Klausur Informatik I
Wintersemester 2009/10
Prof. Dr. C. Damm
24. Februar 2010
von 10:00+x bis 11:30+x Uhr

Vorname:

Nachname:

Matrikelnummer:

Es sind **keine Hilfsmittel** (Skript, Taschenrechner etc.) erlaubt. Papier wird gestellt. Benutzen Sie nur die **ausgeteilten**, zusammengehefteten **Blätter** für Ihre Antworten. Die Blätter **nicht trennen!** Schreiben Sie mit blauem/schwarzem Kugelschreiber. Füller oder Bleistift sind nicht erlaubt.

Viel Erfolg!

	Max. Punkte	Erreichte Punkte
Aufgabe 1 (Allgemeine Kenntnisfragen)	15+0	
Aufgabe 2 (Elementare und strukturierte Daten)	15+0	
Aufgabe 3 (Datenstrukturen und OO-Konzepte)	15+0	
Aufgabe 4 (Sortieren und Suchen)	15+0	
Aufgabe 5 (Algorithmenanalyse)	15+0	
Aufgabe 6 (Iteration und Rekursion)	15+2	
Summe	90+2	

Name: MatNr.:

Aufgabe 1 (Allgemeine Kenntnisfragen [15+0 Punkte])
3P pro Aufgabenteil. Antworten Sie kurz und prägnant.

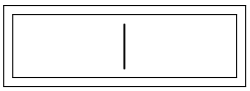
1. Nennen Sie die Voraussetzung zur Anwendbarkeit der binären Suche auf Feldern. Geben Sie in $\mathcal{O}$-Notation die worst-case-Komplexität an (Feldlänge n).

2. Was bedeutet partielle Korrektheit eines Algorithmus in Bezug auf die Problemspezifikation (V, N)?

3. Nennen und erläutern sie das Prinzip, nach dem bei Java die Parameterübergabe bei Methodenaufrufen erfolgt.

4. Welche Elemente einer Java-Klassendefinition werden durch das Schlüsselwort `static` gekennzeichnet? Was bedeutet `private`?

5. Beschreiben Sie das Teile-und-Herrsche-Prinzip und nennen Sie einen Beispielalgorithmus dafür.



Name: MatNr.:

Aufgabe 2 (Elementare und strukturierte Daten [15+0 Punkte])

1. (5P) Stellen Sie den Dezimalbruch 18.625 binär dar.

2. (2P) Sei $b_{N-1}b_{N-2}...b_1b_0 \in \{0,1\}^N$ die N-Bit-Zweierkomplement-Darstellung der ganzen Zahl z mit $-2^{N-1} < z < 2^{N-1}$. Wie erhält man daraus die N-Bit-Zweierkomplement-Darstellung von $-z$?

3. (3P) Wieviele Bytes werden durch folgenden Konstruktoraufruf in Java angefordert? Es genügt eine Zweierpotenz 2^n sowie eine kurze Begründung als Antwort. **int** [][][][] feld = **new int** [16][16][16][16];

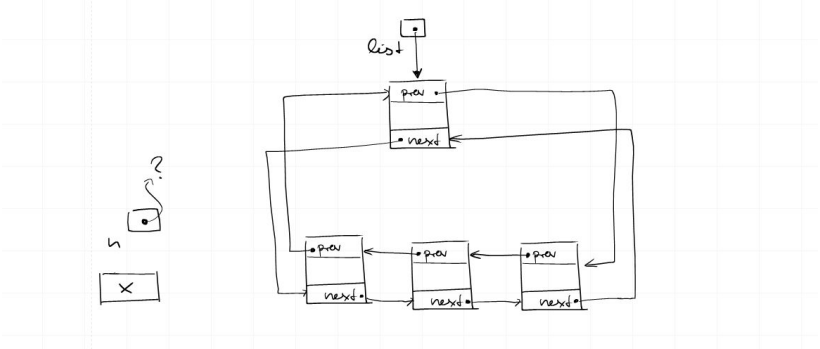
4. (5P) Die `main`-Methode soll einen „Countdown“ erzeugen, d.h. nacheinander „4, 3, 2, 1, Start!“ ausgeben, ist aber fehlerhaft. Beschreiben Sie den Fehler und geben Sie an, an welcher Stelle er auf dem Weg vom Programmcode bis zur Ausführung auftritt. Korrigieren Sie durch *zusätzliche* Programmzeilen (*nachfolgend* angeben, Einfügestelle kennzeichnen).

```
public static void main(String[] argv) {
    int[] x;
    for (int i = 0; i < 4; ++i)
        System.out.println(x[i]);
    System.out.println("Start!");
}
```

Name: MatNr.:

Aufgabe 3 (Datenstrukturen und OO-Konzepte [15+0 Punkte])

1. (6P) Ein Stack für `int`-Werte kann konkret mittels doppelt-verketteter Liste mit Ankerknoten realisiert werden. Dazu eine Zeichnung und Quellcode-Ausschnitte aus Knoten- und Listenklasse:




```
class IntNode{
    int wert;
    IntNode next;
    IntNode prev;
    public IntNode (int x, IntNode next, IntNode prev) {
        wert = x;
        this.prev = prev;
        this.next = next;}
}
```

```
class IntListe {
    IntNode list;
      ...
    void prepend(int x) // Knoten "links" einfuegen
      ...
    void append(int x) // Knoten "rechts" einfuegen
      ...
    void insert(int x)//vor erstem Knoten mit Wert <= x einspeichern
      ...
    int removeHead() // Knoten "links" entfernen (davor Wert liefern)
      ...
    int removeTail() // Knoten "rechts" entfernen (davor Wert liefern)
      ...
}
```

Nennen Sie ein Methodenpaar,das Stack-artiges „Ein-“ bzw. „Ausspeichern“ realisiert. Programmieren Sie davon die Einspeicher-Methode in Java.

2. (4P) Auf eine anfangs leere Queue werden Operationen angewendet. Tragen Sie den Zustand nach den angegebenen Aufrufen so ein, dass das Ende der Queue jeweils ganz links steht (erste Zeile ist komplett vorgegeben). *Deutlich schreiben!*

`enqueue(1);enqueue(2);enqueue(8);`	8	2	1									
`enqueue(5);enqueue(3);dequeue();`												
`enqueue(1);dequeue();dequeue();`												
`dequeue();enqueue(7);enqueue(1);`												

3. (5P) Welche Zeile im folgenden Java-Code verursacht einen Compilezeit-Fehler? Wenn diese Zeile gestrichen wird, welche Ausgabe erzeugt der Programmaufruf `java AB`? Begründen Sie ihre Antworten.

```
1  class A {void hallo(){System.out.print("A");}}
2  class B extends A{String s = "b";void hallo(){System.out.print("B");}}
3  class AB {
4      public static void main(String[] argv) {
5          B y = new B();
6          A x = y;
7          y = x;
8          x.hallo();
9          y.hallo();
10     }
11 }
```

Name: MatNr.:

Aufgabe 4 (Sortieren und Suchen [15+0 Punkte])

1. (7P) Sortieren Sie folgende Zahlenfolge aufsteigend mit Quicksort. Dazu das jeweils letzte Element der aktuellen Teilfolge als Pivot markieren (z.B. einkreisen), ein Schritt des Verfahrens durchführen und das Ergebnis in die nächste Zeile der Tabelle eintragen. Es müssen nicht alle Zeilen der Tabelle benutzt werden.

4	8	6	3	2	7	1	9	5

2. (6P) Zeichnen Sie den binären Suchbaum S, der durch Einfügen von 15, 5, 16, 3, 12, 10, 13, 6, 7 (in dieser Reihenfolge) entsteht. Zeichnen Sie dann den binären Suchbaum, der aus S durch Löschen der 5 entsteht.

3. (2P) Wahr oder falsch?: „Bei Preorder-Traversierung werden die Knoten eines Binärbaumes in umgekehrter Reihenfolge abgearbeitet wie bei Postorder-Traversierung.“ Begründen Sie oder geben Sie ein kleines Gegenbeispiel (mit Angabe und Benennung der Reihenfolgen).

Name: MatNr.:

Aufgabe 5 (Algorithmenanalyse [15+0 Punkte])

1. (6P) Seien m und n natürliche Zahlen. Bestimmen Sie für folgende Programmteile, die GENAUE Anzahl von `tuWas()`-Aufrufen. Der Rechenweg ist Teil der Lösung.
   Geben Sie anschließend in $\mathcal{O}$-Notation eine möglichst gute obere Schranke für diese Anzahl an.

```
13        // Teil A
14        for (int i = 0; i < m; ++i)
15            for (int j = i; j >= 0; --j)
16                tuWas();
```

```
20        // Teil B
21        for (int i = 1; i < m; ++i)
22            for (int j = 0; j <= n; ++j)
23                tuWas();
```

2. (3P) Gegeben sei eine sortierte Folge der Länge n und der Suchschlüssel q. „Ternäre Suche“ hat folgende Rekursionsidee:

   - Ist $n = 1$, so teste, ob q gleich dem einzigen Folgenglied (bei Erfolg liefere dessen Index, sonst -1).
   - Anderenfalls teile in 3 Teilfolgen T_0, T_1, T_2 annähernd gleicher Länge[1], bestimme durch Vergleiche mit dem jeweils letzten Element der 2 Teilfolgen T_0, T_1 in welcher der Suchschlüssel sein müsste und suche in dieser rekursiv weiter.

   Die Anzahl der Vergleiche mit q sei $C(n)$ im schlechtesten Fall. Stellen Sie eine Rekursionsgleichung für $C(n)$ und die Randbedingung $C(1) =?$ auf.

[1] d.h. keine ist länger als $\lfloor n/3 \rfloor + 2$.

3. (6P) Sei n eine beliebige Zweierpotenz. Beweisen Sie, dass für $T(n) = 2n^2 \log_2 n + n^2$ die Rekursionsgleichung $T(n) = 4T(n/2) + 2n^2$ und die Randbedingung $T(1) = 1$ gilt.

Name: MatNr.:

Aufgabe 6 (Iteration und Rekursion [15+2 Punkte])

1. (4P) Die Folge g_n ist rekursiv definiert durch

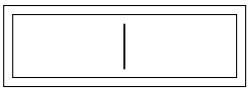
$$g_0 = 0, g_1 = 1, g_n = g_{n-1} + 2g_{n-2}.$$

   Programmieren Sie eine rekursive Methode `int g(int n)`, die zu Argumentwert n das Folgenglied g_n zurückliefert.

2. (5P) Zeichnen Sie den Aufrufbaum zur rekursiven Berechnung von g_5. Stellen Sie damit eine Rekursionsgleichung auf für die Anzahl a_n der Aufrufe, die die rekursive Berechnung von g_n erfordert.

3. (6P) Beschreiben Sie informal eine Idee zur iterativen Berechnung von g_n und programmieren Sie diese.

4. (+2P) Beschreiben Sie informal eine weitere Idee zur iterativen Berechnung von g_n.



Name: MatNr.: